

Graph theory - $C_n(1, a)$

Mykhailo Tsekhmistro, Nejc Zadnik

November 2025

1 Data

$a \backslash n$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
1	=	2	2	2	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
2	=	=	=	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3	=	=	=	=	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
4	=	=	=	=	=	=	=	3	2	2	2	3	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
5	=	=	=	=	=	=	=	=	3	2	2	3	2	3	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
6	=	=	=	=	=	=	=	=	=	=	4	2	2	3	3	2	4	2	3	3	3	3	3	4	3	3	4	4	3	4	3	4	4	4	4	4
7	=	=	=	=	=	=	=	=	=	=	=	=	=	4	2	2	3	3	3	2	4	2	3	3	4	3	3	4	3	3	4	4	4	3	4	3
8	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	5	2	2	3	3	3	3	2	5	2	3	3	4	4	3	3	5	3	3	4	4
9	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	5	2	2	3	3	3	3	3	2	5	2	3	3	4	4	4	3	3	5
10	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	6	2	2	3	3	4	4	3	3	2	6	2	3	3	4	4	4
11	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	6	2	2	3	3	3	3	4	3	3	2	6	2	3	3
12	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	7	2	2	3	3	4	4	4	4	3	3	2	7
13	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	7	2	2	3	3	4	3	3	4	4	3
14	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	8	2	2	3	3	4	4	5	4
15	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	8	2	2	3	3	4	4
16	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	9	2	2	3	3	3
17	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	9	2	2
18	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	10

In this table the column entries are values of n (amount of vertices) and row entries are a (mod connection value).

Here the values under the diagonal are represented in "=" because the graphs are constructed by working with mod values and the graphs are symmetrical and reflexive wrt the middle.

In the first row a lot of values are set to -1 because in that row value of a is set to 1, making the constructed graph a cycle and in cycle graphs the m_2 value is $n/2$ ceiling.

2 Smallest infection number

Lemma 1 (Lemma on infections). *Let n be the number of vertices of the graph $G = C_n(1, a)$. If we infect a cycle in G of the form $(i, i + 1, \dots, i + a, i)$ or $(i, i + a, i + (a + 1), \dots, i + (n - a), i)$, then we infect the whole graph G .*

Proof. We separate the proof into two cases:

- 1st) The cycle is $(i, i + 1, \dots, i + a, i)$. WLOG we use $i = 0$, as the whole graph is symmetric for every vertex. We know that $(0, 1, \dots, a, 0)$ is infected. Now we take a look at the vertex $a + 1$. In this graph, there always exist edges $\{i, i + 1\}, \{i, i + a\}$. So we know that there exist $\{a, a + 1\}, \{1, a + 1\}$ and we know both $1, a$ are infected, as they are part of the initial infected cycle, so $a + 1$ gets infected. We can do this recursively for all vertices and infect the whole graph G .
- 2nd) The cycle is $(i, i + a, i + (a + 1), \dots, i + (n - a), i)$. WLOG we use $i = 0$ for the same reasoning as above. We know, there will always exist edges $\{i, i + 1\}, \{i, i + a\}$ and $\{i, i - 1\}, \{i, i + (n - a)\}$. Let's take a look at what happens with vertices $1, 2, \dots, a - 1$ as the problem is symmetric on another side. 1 has neighbours $0, a + 1$ which are both infected so 1 gets infected. This happens for all vertices $2a, \dots, n - 1, 0, 1, 2, \dots, n - 2a$, same on the right side of vertex 0 . Now let's take a look at vertex $a - 1$. There exist edges $\{a - 1, n - 1\}, \{a - 1, a\}$ and both of those vertices are infected, which means $a - 1$ also gets infected and same for $n - a + 1$. Now we can infect vertices $n - 2a + 1$ and same for $2a - 1$. Now we can infect $a - 2$ and $n - a + 2$. We can do this recursively and infect the whole graph G .

By these 2 constructions, we have shown the lemma. $\square$

Lemma 2. *If no cycle in the graph G can get infected, then G will never get fully infected.*

Proof. Let's assume there exists a graph G' that will get fully infected, but has no cycle that can get fully infected. This means that there exists $v \in \text{cycle}$ that will never get infected but as $v \in \text{cycle} \subseteq G'$ that means that $v \in G'$ which is a contradiction as G' can't be fully infected if it has a vertex that will never get infected, therefore the original statement is true $\square$

Notice that this isn't a converse of the above lemma, as this talks about general cycles.

Theorem 1 (Upper limit of Infection Number). *Let's denote the cycle $c_1 = (i, i + a, i + (a + 1), \dots, i + (n - a), i)$ and $c_2 = (i, i + 1, \dots, i + a, i)$. Smallest infection number in a $C_n(1, a)$ graph is $m_2(C_n(1, a)) \leq \min\{m_2(c_1), m_2(c_2)\}$.*

Proof. By lemma 1, we know

$$\begin{aligned} & \text{we infect the cycle } (i, i+1, \dots, i+a, i) \text{ or } (i, i+a, \dots, i+(n-a), i) \\ & \implies \text{we infect the whole graph } G \end{aligned}$$

Therefore we know, that if we infect one of these two cycles, the whole graph would get infected and we want to infect with as little initially infected vertices as possible, so by now, we were able to lower the upper bound down to

$$m_2(C_n(1, a)) \leq \min\{m_2(c_1), m_2(c_2)\},$$

as either of these 2 cycles will infect the graph and m_2 for a cycle is easily computable. □

Now we can also find a way to determine when each cycle of previous form is shortest, so is the most efficient to use, so we can make our programs more efficient. How we determine that will be shown in the corollary.

Corollary 1. *When $a < \frac{n+1}{3}$, then the shorter cycle will be $(i, i+1, \dots, i+a, i)$, otherwise, the shorter is $(i, i+a, i+a+1, \dots, i+n-a-1, i+n-a, i)$.*

Proof. We can write $n = 2(a+1) + (b+1) - 4$ where $a+1$ is the length of cycle $(i, i+1, \dots, i+a, i)$ and $b+1$ is the length of $(i, i+a, \dots, i+n-a, i)$ cycle. We subtract 4 as we have counted i three times and $i+a, i+n-a$ twice. From the above equality, we can get $b = n - 2a + 1$. Now we just need to solve the inequality $a < b$ as this way, we will figure out when is each cycle longer. Doing this, we get

$$a < \frac{n+1}{3},$$

which proves our corollary. □

Now we can also mention a lower limit, which we believe is true, but cannot prove.

Conjecture 1. *When $\gcd(a, n) = 1$, then*

$$m_2(C_n(1, a)) = \left\lceil \frac{\min(x+y+q+z)}{2} \right\rceil$$

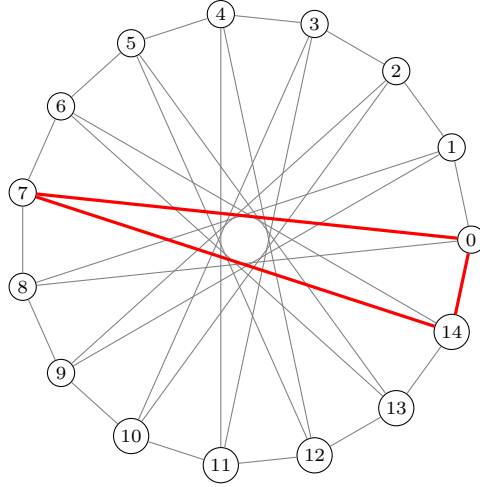
where $x, y, q, z \in \mathbb{N}_0, x+y+z+q \neq 0$ and we get x, y, z, q as solutions to the following Diophantine equation

$$1 \cdot x + a \cdot y + (-1) \cdot q + (-a) \cdot z \equiv 0 \pmod{n}$$

We believe this is true as $x+y+z+q$ represents the length of the cycle in this graph, which represents a "loop" around the main cycle $(0, 1 \dots, n-1, 0)$ of the graph in as little operations as possible (note that we don't need the starting infection cycle to go through each vertex, we are circling the loop as efficiently

as possible).

Here is an example of what was meant for $C_{15}(1, 7)$, where the red triangle represents the fastest circumference, we were talking about before ($7 + 7 + 1 \equiv 0 \pmod{15}$), so by our corollary we have $2a + 1 \equiv 0 \pmod{15}$, so $m_2(C_{15}(1, 7)) = \left\lceil \frac{2+1}{2} \right\rceil = 2$, which is true, can be checked for initially infected set $I = \{0, 7\}$. It has been checked for more cases and we haven't found a counterexample so far.



Definition 1. We should think of fast circling around the graph as going from i back to i with as little moves as possible, where one addition counts as one move. Then backtracking in a cycle is defined as a cycle, where we moved along both 1 and -1 , or both a and $-a$.

Conjecture 2. The most efficient way to infect $C_n(1, a)$ is the one, where no backtracking is involved (so we get a Diophantine equation, where two of the coefficients are equal 0).

Lemma 3. If we want a cycle in the graph, where $a > 2$ and $\gcd(a, n) = a$, to infect the whole graph, at least one edge of the said cycle must be $\{i, i + 1\}$.

Proof. Let's proceed by contrapositive and look at a cycle in G' that doesn't go along any of edges $\{i, i + 1\}$. This means that the cycle only goes along edges $\{i, i + a\}, \{i, i - a\}$. We also know that if $\gcd(a, n) = a$, then the cycle will consist of all $i + ka, k \in \mathbb{Z}$ and the problem is the same for any $i \in \mathbb{Z}_n$, so we can look at $i = 0$, so we are looking at vertices consisting of all multiples of a , and as $a > 2$, this means that the vertices that aren't multiples of a will never get infected. as they have at most 1 neighbour. $\square$

Theorem 2. For $\gcd(a, n) = a$, the construction from lemma 1 is the most efficient way to infect such a graph.

Proof. We look at generators of group $(\mathbb{Z}_n, +)$. Let's take a look at $a \in \mathbb{Z}_n$. $\langle a \rangle$ doesn't generate the whole group $\mathbb{Z}_n$ as $\gcd(a, n) \neq 1$. Therefore we won't be able to infect only moving along $+a$ (look at lemma 3) vertices and we will need to add at least an a amount of operations $+1$ to get to $0 \pmod n$, but that means that this cycle will be longer than the cycle $(i, i+1, \dots, i+a, i)$, so $m_2(C_n(1, a)) = m_2((i, i+1, \dots, i+a, i))$ and in this type of graph the shortest cycle that doesn't "backtrack" (here we are using conjecture 2) is of the form $(i, i+1, \dots, i+a, i)$ and with this cycle, we infect the whole graph. $\square$

Notice that because of the properties of this graph, we will always have the shorter cycle of the form $(i, i+1, \dots, i+a, i)$ as only $a > \frac{n+1}{3}$ will be $a = \frac{n}{2}$ for even n and for $a = \frac{n}{2}$, the other form of cycle doesn't exist, as we don't count a $2 - cycle$ as a cycle here, so we use the longer cycle.

3 Infection numbers on some special cases

3.1 $m_2(C_n(1, \frac{n}{2}))$ for even n

Theorem 3. For $n \equiv 2 \pmod{4}$, the infection number of the graph is $m_2(C_n(1, \frac{n}{2})) = \frac{n+2}{4}$. For $n \equiv 0 \pmod{4}$, $m_2(C_n(1, \frac{n}{2})) = \frac{n+4}{4}$

Proof. In the first part, we will show the fact that $m_2(C_n(1, \frac{n}{2})) \leq \frac{n+2}{4}$. We do this by construction.

Let's denote the initial infection set as I . We have a graph $G = C_n(1, \frac{n}{2})$, so $V(G) = \{0, 1, \dots, n-2, n-1\}$. Our set of edges looks like this for any vertex i : $\{i-1, i\}, \{i, i+1\}, \{i, i+\frac{n}{2}\}$ (remember everything here is modulo n). We then construct I in the following way:

We add every even vertex from 0 to $\frac{n}{2}$ inside I . Note that as $n \equiv 2 \pmod{4}$, $\frac{n}{2}$ is odd. So $I = \{0, 2, 4, \dots, \frac{n}{2}-1\}$. This means $|I| = \frac{n+2}{4}$. Now we show that this set infects the whole graph.

First we look at odd vertices from 0 to $\frac{n}{2}-1$. Their neighbours are $i-1$ and $i+1$ which are even, meaning that all of odd vertices will also get infected. This means we have infected all vertices from 0 to $\frac{n}{2}-1$. Now we take a look at the vertex $\frac{n}{2}$. Among it's neighbours are 0 and $\frac{n}{2}$ which are both infected, meaning that $\frac{n}{2}+1$ also gets infected. We can do this recursively for $i+\frac{n}{2}$ as we can always infect $i+\frac{n}{2}-1$ and i is infected from before. This way we infect all vertices. Therefore, we have shown the first part.

In the first part, we have basically just constructed a way to infect a cycle of the form $(i, i+1, \dots, i+a, i)$ and by theorem 1 and conjecture 2 we assume, that this is the most efficient way to infect the graph (note that we don't count a 2-cycle as a cycle here, so the cycle $(i, i+\frac{n}{2}, i)$ isn't valid).

Now we look at graphs with number of vertices $n \equiv 0 \pmod{4}$. We find the cycle $(i, i+1, \dots, i+\frac{n}{2}, i)$ which is an odd cycle. By theorem 1 and conjecture 2 we infect this graph, by infecting the before mentioned cycle and an odd cycle is infected with infection number $\frac{n+4}{4}$. $\square$

3.2 Infection number for $a \in \{2, 3, n-3, n-2\}$

Theorem 4. If $a \in \{2, 3, n-3, n-2\}$ in graph $C_n(1, a)$, then $m_2(C_n(1, a)) = 2$.

Proof. Let us first take a look at the case, where $a = 2$. Then our shortest cycle will be $(i, i+1, i+2, i)$, which is a 3-cycle so we can infect it with 2 infected vertices. By theorem 1, we will therefore infect the whole graph, therefore $m_2(C_n(1, 2)) = 2$. We know that the smallest possible m_2 is 2, which means that this will definitely be the most efficient way, so equality holds.

We can use the same argument for $a = 3$ as we then have a 4-cycle, which we can also infect with 2 infected vertices, meaning that $m_2(C_n(1, 3)) = 2$.

The problem is symmetric for pairs $n-2, 2$, and $n-3, 3$. $\square$

4 Script explanations

Note: You can find the whole Unity project as well as the application uploaded to GitHub at the following link: [QuarticM_2 computations](#).

The script has 8 different scripts responsible for unique things that vary in importance, but most importantly for this report I will divide the scripts into I) "compm2v3.cs" where I will describe it more because it is the most significant script that is responsible for computing m_2 in my program, and II) where I briefly explain the other scripts which are much less important to the topic and mostly responsible for the visuals on the UI.

4.1 M_2 computation, "compm2v3.cs"

The main script in this program is called "compm2v3.cs" it is responsible for computing value of m_2 for the current graph as well as infection visuals. The computations of m_2 is done in the following way:

Start from $k = 2$, generate all subsets of the graph vertices that contain the initial vertex index 0 (we generate these sets w.r.t. symmetry). Then for all these subsets check if any result in the entire infection of the graph. If so then return this subset of vertices and infect them. If no such subset for current k is found then we iterate through next k . As we found there is a value of k for which any quartic graph is completely infected, that is $\lceil \frac{a+1}{2} \rceil$, so our k iterates up to this value.

All the computations described above are done in the function of the script called "ComputeM2.Optimized", as you can imagine for large values of n and a the process of generating the subsets of size k for a graph and then checking if that graph is infected or not is really recourse intensive, although I have tried to implement a lot of optimization to my script, this process is still very rough and is a "brute force method" unfortunately.

My script has a manager called "ComputeM2Button" whose role is to help working with the UI as well as the computation part. When the computation of m_2 is called, this manager calls for the computation of "ComputeM2.Optimized" this function is run on a separate thread that allows us to use the program while the m_2 is computing. In the script we have a variable "workerK" that is a small bridge to pass a value of K that the script is currently generating/checking the graphs on. This value of K is later passed onto the UI element.

Here I will go through the optimization methods that I used in the process of computing m_2 :

1. PassesReflectionCanonical()

This method is a check for reflections. The script takes in the set of vertices representing a generated set of size k , and a value of n that is the

size of the graph. It returns true if the set is canonical, and false if the set is a mirror image and we have already seen this set. The method looks for the distance from the zero element to the first infected vertex going clockwise and then the first infected element counter clockwise, that is `set[1]` and `set[k-1]`. Then if the clockwise distance is shorter then keep the set, otherwise skip it.

2. Pre verification filter

Before running a full, expensive simulation (BFS), the script performs a check to see if the infection can even move past the first step.

If no healthy vertex in the entire graph has at least 2 infected neighbors at the very start, the infection is impossible from the start.

3. BFS with Stamp

BFS requires clearing arrays (like `visited[]`) between every run. Clearing an array of size n is intensive for large combinations.

Stamp: Instead of a `bool[]` that needs to be reset to false, the script uses an `int[]` `infectedStamp` and a stamp counter.

To check if a node is infected, it checks if `(infectedStamp[u] == stamp)`.

To "clear" the whole graph for the next combination, the script simply does `stamp++`, this clears $O(n)$ array clearing operations for every combination.

4. Memory/Allocation Management

Variables like `infNbrCount`, `q` (the BFS queue), and `stepSnapshot` are created once at the start of the computation. The worker thread then reuses these same memory slots thousands of times.

5. Lexicographic generation

Instead of using complex "nested" functions or trying to save every possible group of vertices at once which would crash the program by using too much RAM-the script uses a simple "odometer" logic.

The script looks at the current set, finds the last number that can still be increased, increases it up by one, and resets the numbers next to it. This allows the script to cycle through every possible combination one by one using a single piece of memory. Because it just overwrites the same small list over and over instead of creating new ones, the memory usage decreases.

This script is also responsible for the infection process. Once the graph is generated and we also computed the value of `m2` for the current graph, then exactly `m2` vertices that result in complete infection of the graph turn red and their status is set to `"infected" = true`.

We then have an option to infect the whole graph, if a toggle named "Auto infect" is on, then once we press the "infect" button the script will begin a routine that runs every 0.3 seconds that will execute one step of infection in the graph. As desired, in each step we check if a healthy vertex has 2 infected

neighbours, if so then we infect that vertex, we perform this step for all healthy vertices simultaneously. This continues until the whole graph is infected. When the toggle "Auto infect" is set to off then we don't start a coroutine but perform only one single step of infection at a time.

4.2 vertex.cs

The script named "vertex.cs" is the second script in terms of importance, this script is assigned to a vertex prefab and holds information about the vertex, namely: index in "index" where we assign all generated vertices indices from 0 to n-1, set of neighbours of the vertex in "nbrs" and a boolean toggle to check if a vertex is infected. All this information is then accessed from other scripts that make the visual infection of the graph.

Other than these three variables, the "vertex.cs" script also contains methods and helpers that are responsible for the "highlighting" mechanic of the application.

Each vertex has a main visible circle sprite, and an invisible outer layer sprite. When a user clicks on a vertex, if it is not highlighted, then we take a random colour from "highlightMaterials" list and apply it to the highlighting child of the vertex, then we remember this colour and for every other non-highlighted neighbour of the vertex that we initially clicked on, we set their highlighting colour to match the first one.

4.3 highlighter.cs

"highlighter.cs" is directly connected to the "vertex.cs" script and serves two purposes, first is to be a raycaster, on click we shoot a ray from camera to where the mouse is pointing on the screen, if the raycast returns object tagged "generated" (this tag put on only the vertex prefabs) then we send a method request on the vertex we hit for "HighlightSelfAndUnhighlightedNeighbors()".

The second purpose is to have a button method for resetting all colours of highlighted vertices. When we highlighted some vertices and no longer wish to highlight them, this method is called when the button "reset_colours" is pressed, it goes through every generated vertex and sets their highlighting material to be "default".

4.4 gen_vertices.cs

This script generates vertices from the two input fields for "n" and "a". This script generates n vertices in a circle around the center reference point, depending on n this script alters the size of each vertex and the distance from each vertex from the center.

For small values of n we have larger vertices, and for bigger n the size of each vertex is small.

Formula for rescaling vertices is

```
Mathf.Clamp((0.8f / Mathf.Sqrt(n)) * sizemod, 0.01f, 1f);
```

`Mathf.Clamp` ensures the vertices dont get too small and disappear when `n` is too large. values `size mod` is just a constant I used to make the altering more precise.

Formula for the distance from each vertex to the center (radius) is

```
float radius = Mathf.Sqrt(n) * 0.5f;
```

After the script is does generating all `n` vertices it calles for the next script to generate lines for all connected vertices.

4.5 `gen_lines.cs`

This script is called once right after all the vertices are generated. When we generate all vertices, we put them all in a list of "allVertices" and then pass it to the `gen_lines` script. Unity has a component called "Line Renderer", this component is very useful for 2d environments, for every vertex that we generate, we take a vertex and for each of its neighbours $\{+1, +a\}$ we pass coordinates of the pair and this component "line renderer" draws a line in between their positions.

These lines serve only visual purpose.

4.6 `customInfect.cs`

This script works when the graph is generated. The user needs to fill in the input field named "custom infect", for each vertex that the user wants to infect you need to type its index and separate all indexes with a coma.

Example: "0,1,3,4,6,8" in this example vertices indexed 0 1 3 4 6 8 will be set to infected in the graph.

This infection sets the toggle of each of the desired vertex to be "infected" = true. The infection process is implemented in the "computem2v3.cs".

4.7 `loading_circle`

This is a very simple script, it only takes a gameobject reference for the loading circle and constantly rotates it. It is usually invisible but when the computation of `m2` is running we set it to be visible, and once the computation is done we again set it to be invisible.